

Qué es un agente en este curso

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES Sesión 5 de 12 · Notas en español (es-ES)

Una pregunta larga sobre la ventana de contexto («esta generación lleva eternidades: ¿cómo se gestiona el contexto?») empujó la sesión 5 a la definición de trabajo de **agente**. La generación de código no es una llamada al LLM. Es un **bucle agentic**: un agente principal (orquestador) escribe una lista de to-dos; los **subagentes** ejecutan trozos con su propia ventana de contexto, prompts, tools, memoria, skills y, opcionalmente, su propio modelo y modo de permiso. Claude Code es el cliente agentic de referencia; el examen Foundation más adelante pide sistemas parecidos con las APIs de Claude.

LOS DOCS Y LAS LISTAS DEL MARKETPLACE CAMBIAN

Los subagentes integrados (Explore, Plan, ayudantes de status-line) y la lista oficial de tools (Glob, Grep, Read, Write, ...) se reordenan en la documentación. El formador tuvo que buscar en directo una lista vertical. Usad **Claude Code**, **tool use** y **Managed Agents** para los nombres vigentes.

Agente principal frente a subagente

En Claude Code hay un **agente principal**: la sesión donde escribís. Orquesta y planifica. Un **subagente** ejecuta una tarea. Palabras del formador: «agent no es más que un sub-agent en el caso de Claude» cuando habláis de los markdown especializados que añadís. Hay un agente principal y muchos subagentes. Pueden dispararse solos cuando un prompt encaja con su *description*, o los invocáis. Cada uno corre en su propia ventana de contexto con system prompt a medida, acceso propio a tools y permisos independientes.

Pieza	Agente principal (sesión Claude Code)	Subagente
Trabajo	Orquestar: memoria + skills + contexto → lista de to-dos	Ejecutar un trozo (carpetas, backend, frontend, Docker, revisión)
Contexto	El chat que veis; puede crecer mucho	Ventana nueva acotada al trozo
Ficheros de memoria	El CLAUDE.md global siempre entra	Global más el CLAUDE.md de carpeta que coincida (backend frente a frontend)
Borrador a corto plazo	Sandbox; se descarta al terminar el trozo; no lo controláis	La misma idea, local a esa ejecución

La memoria a largo plazo son ficheros, no el chat

Rajesh preguntó si la «memoria» vivía en el sandbox con un TTL corto. El formador separó: los CLAUDE . md del repo git son **largo plazo** y físicos (global, backend/CLAUDE . md, frontend/CLAUDE . md). El estado del sandbox que cose funciones una a una es **corto plazo** y se descarta. CLAUDE.md es «configuración maestra, no un agente maestro». Los ficheros anidados no crean orquestadores extra; son qué paquete de instrucciones carga un agente dado.

Tres agentes en un trabajo full-stack pueden correr con tres mezclas: (1) solo global (crear carpetas), (2) global + backend, (3) global + frontend. No es un mensaje enorme. Así una generación de horas cabe en los límites de contexto: trozos, planificados, y luego enviados.

Cuatro configuraciones que todo agente tiene

Sea principal o sub, el formador listó cuatro ingredientes siempre presentes, y extras que fijáis en el fichero del agente:

1. Prompt (el texto de sistema / rol del agente).
2. Contexto (lo que hay en la ventana de esta ejecución).
3. Tools (integradas y/o MCP — p. ej. ECR en vez de Docker Hub para imágenes base).
4. Memoria y skills (qué CLAUDE.md y qué paquetes SKILL.md).
5. Opcional: modelo, modo de permiso, max turns, tools denegadas, hooks, aislamiento, color, prompt inicial, effort, background.

Campos obligatorios al escribir un subagente: **name** y **description**. La description es cómo enruta el orquestador («revisa el código por calidad... úsalo justo después de escribir o modificar código»). Las tools pueden ser una lista estrecha (solo Read → el revisor no puede Write) o un comodín si de verdad no lo sabéis. Las tools a medida son tools MCP, no una función local cualquiera haciéndose pasar por tool (la aclaración fue explícita).

Dónde viven los ficheros de agente

- Proyecto: carpeta `.claude/agents` (el formador: «dot cloud slash agents») en el repo — se clona con el proyecto.
- Perfil de usuario (Windows / macOS / Linux): disponible para todos los proyectos de ese usuario.
- Plugins: empaquetados con hooks y skills (módulo 06).
- Flags de CLI si lanzáis agentes desde la línea de comandos.
- Integrados: Explore (tools orientadas al sistema de ficheros), Plan, status-line, y lo que listen los docs actuales — se usan automáticamente.

Modelos sin estado, orquestación con estado

Un alumno esperaba que CLAUDE.md y las skills se «cachearan» como palabras repetidas del prompt. El formador: los LLM son **stateless**; una petición puede caer en otra instancia (escalado horizontal). La caché de prompts es una función explícita de API posterior — no algo que se asuma en Claude Code. Los *resultados* de tools sí pueden cachearse en infraestructura compartida (mencionó cachés tipo Redis) y reutilizarse entre instancias. El historial del chat lo guarda el *cliente* / *orquestador*, por eso podéis reanudar una lista de to-dos. Varios subagentes de un proyecto se coordinan por ese orquestador, no charlando entre iguales, salvo que el agente principal pase artefactos (ficheros, OpenAPI, tipos).

Destino de examen

«Implementar algo como Claude Code» con el agent SDK / APIs de Claude es el norte de Foundation. En este módulo basta el vocabulario: orquestador, subagente, ventana de contexto propia, delegación por description, allow/deny de tools, heredar o fijar modelo, plan editable y luego congelado. Demasiados CLAUDE.md solapados pueden alucinar — aviso de Pedro; la mitigación enseñada es forzar un paso de aclaración en la memoria global antes de generar.

Documentación oficial

- **Visión general de Claude Code**

<https://code.claude.com/docs/en/overview>

Subagentes, agentes integrados y ámbito proyecto frente a usuario.

- **Inicio rápido de Managed Agents**

<https://platform.claude.com/docs/en/managed-agents/quickstart>

Agentes de plataforma cuando salgáis del cliente IDE.

- **Visión general de tool use**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Tools estilo Read/Write/Grep y tools MCP en un agente.

- **MCP**

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

Cómo se limita un agente a ECR frente a Docker Hub en el ejemplo Docker.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.